

Group 39

https://github.com/MarlyB5/DataStructures_Project.git

Introduction

Data structures form one of the most crucial concepts in Computer Science. Their efficiency directly influences the performance of any operation that involves insertion, search, deletion, and other related tasks. Although there are many ways to ensure efficiency, some require the use of complicated algorithms to achieve certain characteristics. For instance, a balanced binary search tree, such as the AVL tree, guarantees perfect balance but involves increased complexity.

Randomised data structures present another option that enables efficient implementations without compromising simplicity. In most cases, they achieve their results through the utilisation of probabilistic algorithms. As a result, their performance remains consistent, although the actual outcomes may vary based on input types. Among the commonly used randomised structures is the Treap. It represents a randomised binary search tree that consists of keys and priorities. While each node contains a binary search tree key that ensures correct order during insertion, the priority field holds a randomly generated value that creates a min-heap property.

The current research project aims to implement a Treap in Java. The structure will be compared to an AVLTreeMap and the TreeMap from Java's libraries to identify differences and similarities in terms of operations and efficiency. Additionally, the project seeks to develop an algorithm that utilises the implemented data structure for sorting purposes and benchmark it against common sorting methods. The current report discusses the implementation and comparison of the developed algorithms with others.

The purpose of this document is to explore the efficiency of the Treap in different scenarios. The project includes the implementation of the generic Treap class in Java, the development of the TreapSort algorithm, and the evaluation of both against other data structures and sorting methods.

Background

The Treap data structure consists of a binary search tree with additional heap properties. Every node in a Treap stores two parameters: a key and a priority. Keys define the binary search tree ordering. According to this property, any element within the left subtree is smaller compared to the current root, and all elements within the right subtree are larger. Priorities are random numbers that help with preserving the heap orderings.

The Treap fulfils two sets of properties at the same time. First, it is a binary search tree in terms of key orderings. Second, all nodes follow either a min-heap or a max-heap pattern in terms of their priorities, meaning that a parent node always has a priority higher or lower than its child's

priorities. Due to such a design, a Treap tends to be self-balancing, thus having an expected complexity of $O(\log n)$.

Search procedures in the Treap work identically to how they work in a regular binary search tree. By providing a key, one can traverse through nodes by performing comparisons of key values in nodes and making left or right turns accordingly. Therefore, the expected time of the searching operation is logarithmic.

When it comes to inserting elements, there are two main steps in this procedure. At first, a new node is placed into the tree according to the binary search tree requirements. In such a way, a situation in which the new node violates the heap property occurs. For that reason, rotation operations must be done to place the new node in the correct location based on priority orderings.

Deletion operations can be considered as reverse insertion operations in that sense. In other words, the element needs to be found, after which rotations can be applied in accordance with the priorities of the node's children. Rotation operations continue until the node becomes a leaf in the tree. At this point, it can be deleted from the tree.

As mentioned before, the main difference between a Treap and other self-balancing binary search trees lies in the use of random numbers in the form of priorities. In the case of other types, such as an AVL tree, balancing is implemented deterministically. Consequently, such an approach leads to more complex implementations, whereas using probabilities makes the algorithms much simpler. However, the actual shape of the tree may differ between runs.

Implementation - ADT Methods, Method Implementation and Testing

Design Choices

We chose to implement the Treap as a max-heap. This decision does not have any effect on the performance of the Treap; we just considered it more natural. Originally, nodes were implemented in an array-like fashion, similarly to a heap. This caused a lot of problems and memory inefficiency, as the Treap is not filled in a left-to-right fashion, causing a lot of null entries. This made us change the nodes to be linked-list-like, where a pointer to the root node is stored in the Treap and then every node stores pointers to its parent and children.

Priority is randomly chosen at the creation time of the node. The generator is stored as an instance variable in the Treap itself. This was chosen so that the generator can be given a specific seed to reproduce a certain result. Originally, the range of values chosen was the entire range of integers. We then restricted this to 0-255, to make the priorities easier to read when the Treap is printed. We later changed this back to the full range to ensure it did not affect performance during our testing.

ADT Methods

The following public methods were implemented for the Treap:

- **get()** searches for a key and returns its value, or null if the key isn't found

- **insert()** and **remove()** insert or remove a node with the given key into the Treap
- **toString()** returns a string representation of the Treap
- **containsKey()** works in a similar way but returns a boolean instead
- **firstKey()** and **lastKey()** return the smallest and largest keys by traversing left or right until reaching a null node. This works because of the Binary Search Tree property
- **size()** and **isEmpty()** use a counter that updates on every insert and delete, rather than counting nodes each time
- **inOrder()** traverses left, then current node, then right, which gives the keys in sorted order from smallest to largest.
- **height()** recursively finds the longest path from the root to a leaf, returning minus one if the tree is empty

Searching

Searching presented a unique challenge to the Treap. Searching for an existent value can be done easily with a binary tree search, but insertion relies on knowing the position the node should be in. To minimise code repetition, we wanted to have one method which fulfils both purposes. To do this, we created a new `SearchResult` class. If the node existed, the `SearchResult` would contain the value stored at the node (for the `get()` method) and a pointer to the node itself (for insertion). If the node did not exist, the `SearchResult` would contain the parent node and whether the key would become the left or right child node. This promoted code reuse with little added complexity.

Rotation

Rotation is needed to maintain the heap invariant. We created a `rotateLeft()` and `rotateRight()` method which follow a similar but slightly altered set of steps. Once we found a pseudocode algorithm online, it was easy to implement. We thought recursion would be needed, but there is a fixed number of steps to rotate a subtree of any size. One important step that caused some bugs at the start was that the newly rotated subtree needed to be reconnected to the rest of the tree, which was a step we had to reason about, as it was not described in the pseudocode algorithm.

Printing

Printing is implemented through a modified version of the `BinaryTreePrinter` class we were given for use in our weekly assignments. We changed it slightly to work with Treaps, and we also made it print the priority of the node in brackets next to its key.

Insertion

Insertion into the Treap follows three steps. First, a `SearchResult` object is generated from the provided key. If it finds a node with the key, we simply update its value. This allows us to avoid messing with the binary search tree and heap invariants. If the key is new to the Treap, we use the 'hypothetical position' information stored in the `SearchResult` to make the new node adhere to the binary search tree invariant. Finally, we rotate the new node up the tree where necessary to make sure the heap invariant is maintained.

Deletion

The delete() method is more complex than the other methods.

Instead of removing a node directly (which would break the tree structure), the node is rotated downwards until it becomes a leaf and then removed.

Each rotation direction depends on which child has the higher priority: right rotation if the left child's priority is higher, left rotation if the right child's priority is higher.

If only one child exists, the rotation is done in that direction.

Testing

The tests covered both normal and edge cases, including:

- Duplicate key insertions
- Empty tree behaviour
- Deleting keys that don't exist
- Deleting leaf nodes and nodes with two children
- Ordering checks to verify firstKey() and lastKey() return correct values after insertions and deletions

Two bugs were found during testing:

1. A NullPointerException in the search method - when the tree was empty, it tried calling getKey() from a null reference, which caused insert and search operations to crash.
2. An infinite loop in delete was caused by the rotation methods not updating the root node's parent pointer correctly, so the node never actually moved.

Both issues were found through unit testing and were fixed before the final submission.

Experimental Design

We ensured the algorithms were tested under fair and controlled conditions to ensure a fair comparison. Each algorithm was tested using.

- Identical input sizes
- Same hardware and runtime environment
- Same measurement unit (ns)

This ensured that the performance differences were not due to external factors.

However, the experimental evaluation had its limitations, as Java does not execute code directly; it instead runs in the JVM. This introduced just-in-time compilation, which causes code to be optimised during runtime. Java also automatically manages memory, which can pause execution and introduce unpredictable delays. Timing also depends on CPU load and background processes. Although the results were mostly in line with our theoretical analysis

Data Structure Analysis

Input data selection

Four input types were used to evaluate the algorithm's behaviour:

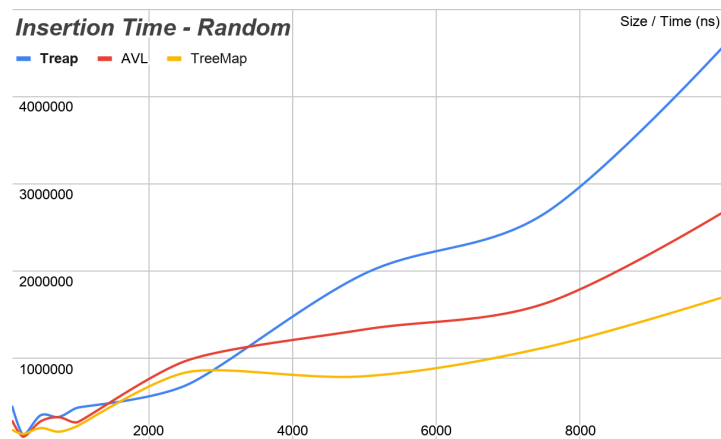
- **Random input** → represents average-case behaviour
- **Sorted input** → tests best/worst-case scenarios
- **Reverse input** → stresses pivot-based algorithms
- **Partially sorted input** → simulates real-world data

This selection is important because algorithm performance is often **input-dependent**, not just complexity-dependent.

structure	operation	input	size	time_ns
Treap	insert	sorted	100	64559
Treap	insert	sorted	500	645340
Treap	insert	sorted	1000	216325
Treap	insert	sorted	2500	497244
Treap	insert	sorted	5000	609864
Treap	insert	sorted	7500	760108
Treap	insert	sorted	10000	1045831

Graph showing the difference between insertion time vs the input size of the data

Input size scaling



We tested across a large amount of varying input sizes. (100 → 10000). This allowed us to observe the time complexity, allowing us to validate our theoretical time complexity.

Execution time was measured in nanoseconds

Due to JVM optimisation and memory effects, trends at larger input sizes are usually more reliable indicators of algorithm efficiency.

Overall, the method by which we got results provided a valid comparative framework, but the results should be considered along with the theoretical expectations.

Experiment Results:

Overall, the general trends were observed

- TreeMap consistently performed best overall
- Treap exhibited higher overhead and more variability
- AVL Tree Map showed stable and predictable performance

TreeMap Analysis:

TreeMap is implemented using the Red-Black Tree, which ensures $O(\log n)$ for insertion, search and deletion. The Red-Black tree is a type of self-balancing Binary Search Tree which ensures the longest path is at most 2x the shortest path.

The TreeMap performance was consistently strong across all the input types due to:

- The struct balancing rules
- Minimal restructuring
- Highly optimised implementation

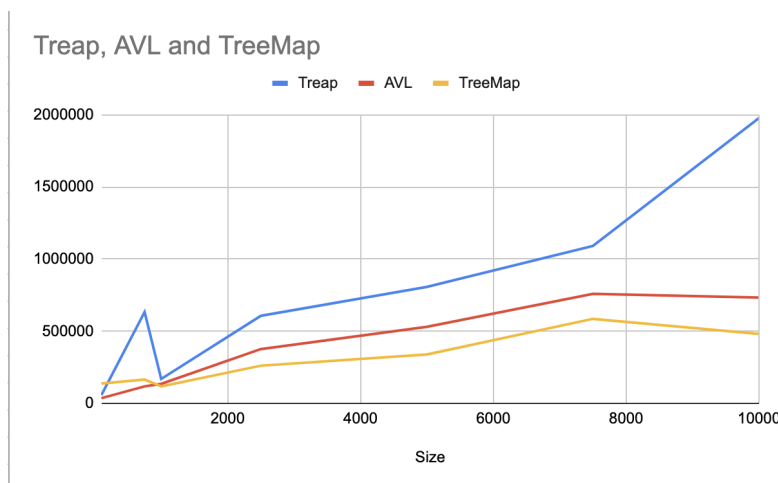
AVL TreeMap Analysis:

AVL trees maintain strict height balance as the difference between the left and right subtree ≤ 1 .

This ensures:

- very fast searches
- predictable $O(\log n)$ performance

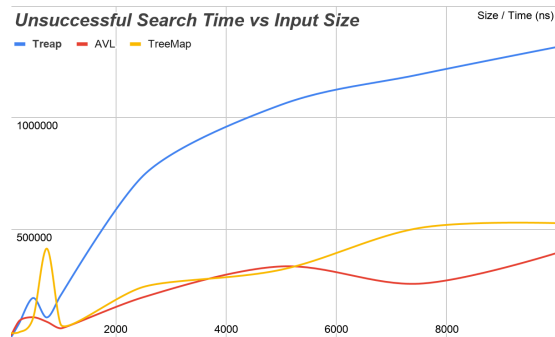
However, AVL trees require **frequent rotations** during insertion and deletion, which increases overhead.



Graph displaying the performance of removal of sorted data

Key Observations so far:

- AVL Treemap is slower for insert/delete operations
- AVL Treemap can be competitive for search operations



Graph comparing unsuccessful search time versus the input size. AVL Treemap ends up working just as well as the TreeMap for search methods.

Treap Analysis:

Treaps combine both the Binary Search Tree ordering and the Heap ordering.

This leads to

- Expected $O(\log n)$ performance
- The random priorities lead to unguaranteed balancing

The treap performs slower due to additional rotations to maintain the heap property, higher constant factors and the random priorities cause less predictable structures.

structure	operation	input	size	time_ns	<Table comparing the removal time.
Treap	remove	random	10000	3692700	
AVL	remove	random	10000	2219518	
TreeMap	remove	random	10000	1639392	

The Treap performs much worse than the TreeMap and only marginally worse than the AVL Treemap.

Unlike the sorting algorithms, the tree-based structures are less sensitive to input order as they balance dynamically and the operations depend on the structure rather than it's initial ordering.

Summary of Findings:

- The TreeMap is most efficient in practice
- Treap is simpler but less efficient
- AVL Treemap is efficient when searching, but performs worse for insertion and deletion.

Sorting Performance Analysis

This project evaluated and compared the performance of 5 sorting algorithms

- TreapSort
- PQSort
- QuickSort
- MergeSort
- JavaSort

The aim was to analyse the performance under different conditions to identify how the theoretical complexity aligns with the practical results.

Background Theory

Algorithms are analysed using Big-O notation, which describes how execution time grows with input size.

Key complexities:

- $O(n \log n)$ → efficient and scalable algorithm (MergeSort, HeapSort, JavaSort)
- $O(n^2)$ → inefficient for large inputs (worst case QuickSort)

The theoretical complexity provides a good guideline for our expectations for performance, but the actual performance depends on factors such as input distribution, implementation details and system-level behaviour.

The experiment structure was very similar to the structure used for comparing data structures

Four input distributions were used:

- **Random** – unordered data
- **Sorted** – already sorted data
- **Reverse** – descending order
- **Partial** – partially sorted data

These were chosen to evaluate both average-case and worst-case scenarios. Algorithms are also input-dependent.

QuickSort Analysis

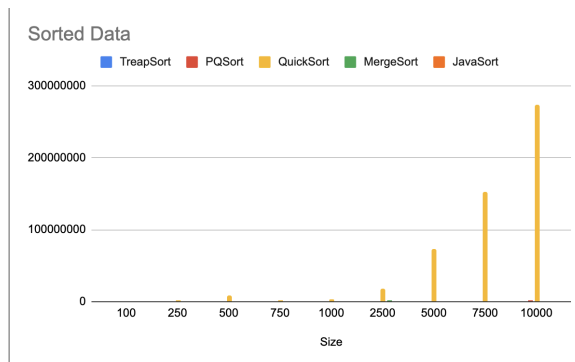
QuickSort performed well on random input but showed extremely poor performance for sorted and reverse inputs.

This behaviour is due to the pivot selection strategy. The implementation selects the last element as the pivot. When the input is already sorted, this results in highly unbalanced partitions, causing the recursion depth to increase significantly.

As a result, the time complexity degrades from $O(n \log n)$ to $O(n^2)$.

structure	operation	input	size	time_ns
Sorting	QuickSort	reverse	100	200919
Sorting	QuickSort	reverse	250	2425581
Sorting	QuickSort	reverse	500	7016068
Sorting	QuickSort	reverse	750	1465514
Sorting	QuickSort	reverse	1000	2380418
Sorting	QuickSort	reverse	2500	12389224
Sorting	QuickSort	reverse	5000	47689614
Sorting	QuickSort	reverse	7500	105546758
Sorting	QuickSort	reverse	10000	188006109

A table highlighting the time taken to sort the reversed data



Graph displaying the worst-case scenario for quicksort using sorted data

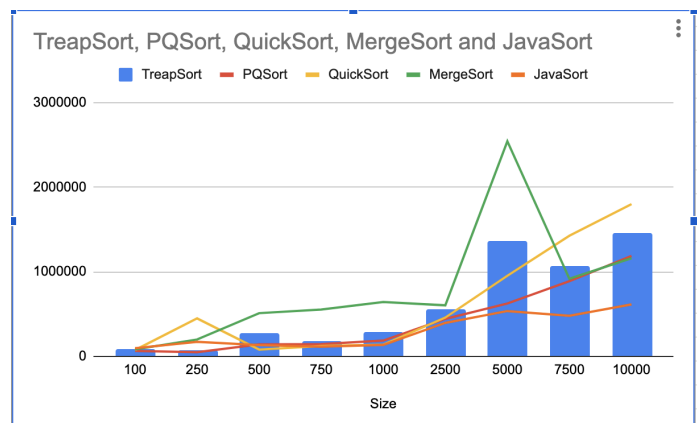
JavaSort Analysis

JavaSort consistently delivered the best performance across all input types.

This is due to its use of **TimSort**, a hybrid algorithm combining merge sort and insertion sort. TimSort is adaptive and can detect existing order within the data, allowing it to perform significantly fewer operations on partially sorted inputs.

This allows for low execution times for partially sorted data.

Graph showing the performance of Javasort on partially sorted data. JavaSort performs significantly better. >



PQSort Analysis

PQ sort uses a priority queue to sort elements.

The performance remains stable across all the input types because the heap operations do not depend on the initial ordering. Although due to repeated insertions and removals, it is slower than other sorting algorithms such as merge sort.

TreapSort Analysis

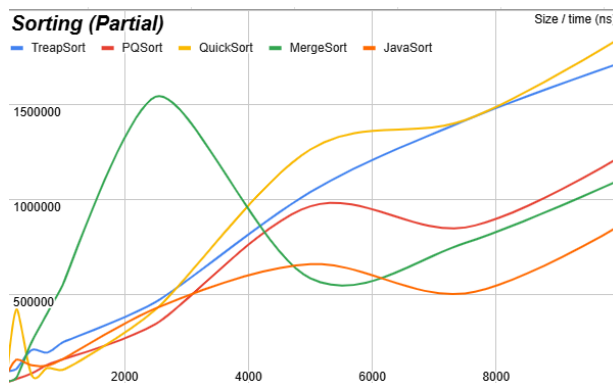
TreapSort combines binary search tree ordering with heap-based balancing.

Although it achieves expected $O(n \log n)$ performance, it is slower in practice due to:

- rotations during insertion
- maintaining two structural properties
- Higher constant overhead

MergeSort Analysis

MergeSort demonstrated stable performance across all inputs.



Since it always divides the data evenly, its time complexity remains $O(n \log n)$ regardless of input order. This makes it highly reliable but slightly slower than JavaSort due to additional memory usage and merging overhead.

Conclusion

Treaps are a viable data structure with their own niche. While they do not strictly ensure a maximum height of $\log(n)$, the result is generally acceptable in practice.

Insertion is typically quicker in Treaps than in other self-balancing trees, as fewer rotations are required. Because of this, Treaps are best suited for dynamic data where insertions and deletions are frequent, but better search performance than a standard binary tree is still needed.

A different self-balancing tree implementation, such as an AVL tree or Red-Black tree, would be better suited to larger, static datasets where search efficiency becomes a more important metric.

TreapSort is not a viable Sorting algorithm. Since every node must be traversed, the heap property offers no advantage and instead introduces additional overhead during insertion. In this case, simpler structures or standard algorithms such as MergeSort or QuickSort would perform better.

Overall, Treaps provide a good balance between simplicity and performance, but are not always the best choice depending on the use case.